Although it might be a useful technique for debugging purposes, in a production environment, it is better to leave it disabled, to reduce the database request networking overhead.

8.4.3 Logging parameters

Either the JDBC `Driver` or the `DataSource` must be proxied to intercept statement executions and log them along with the actual parameter values. Besides statement logging, a JDBC proxy can provide other cross-cutting features like long-running query detection or custom statement execution listeners.

8.4.3.1 DataSource-proxy

A lesser-known JDBC logging framework, datasource-proxy[9] provides support for custom JDBC statement execution listeners. In Java EE, not all application servers allow configuring an external `DataSource`, as they rely on their own custom implementations that bind the user supplied JDBC `Driver`. Because it can only decorate a `DataSource`, datasource-proxy might not be suitable for all Java EE environments.

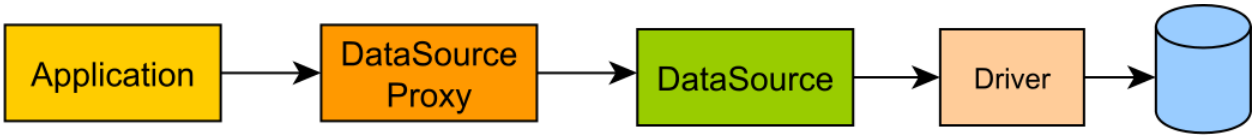

Figure 8.4: DataSource-Proxy Architecture

On the other hand, the programmatic configuration support fits the Java-based configuration approach taken by most modern Spring applications:

```
@Bean
public DataSource dataSource() {
    SLF4JQueryLoggingListener loggingListener = new SLF4JQueryLoggingListener();
    loggingListener.setQueryLogEntryCreator(new InlineQueryLogEntryCreator());
    return ProxyDataSourceBuilder
        .create(actualDataSource())
        .name(DATA_SOURCE_PROXY_NAME)
        .listener(loggingListener)
        .build();
}
```

[9] https://github.com/ttddyy/datasource-proxy

In the following example, datasource-proxy is used to log a batch insert of three `PreparedStatement(s)`. Although normally a batch is printed in a single line of log, the output was split into multiple lines to fit the current page layout.

```
Name:DATA_SOURCE_PROXY, Time:6, Success:True,
Type:Prepared, Batch:True, QuerySize:1, BatchSize:3,
Query:["insert into post (title, version, id) values (?, ?, ?)"],
Params:[(Post no. 0, 0, 0), (Post no. 1, 0, 1), (Post no. 2, 0, 2)]
```

Not only the bind parameter values are now present, but, because they are grouped all together, it is very easy to visualize the batching mechanism too.

With the custom statement listener support, datasource-proxy allows building a query count validator to assert the auto-generated statement count and, therefore, prevent N+1 query problems during the development phase.

8.4.3.2 P6Spy

P6Spy[10] was released in 2002, in an era when J2EE application servers were ruling the world of enterprise systems. Because Java EE application servers do not allow programmatic `DataSource` configuration, P6Spy supports a declarative configuration approach (through a `spy.properties` file).

P6Spy offers support for proxying both a JDBC `Driver` (which is suitable for Java EE applications) or a JDBC `DataSource` (supported by some Java EE containers and common practice for Spring enterprise applications).

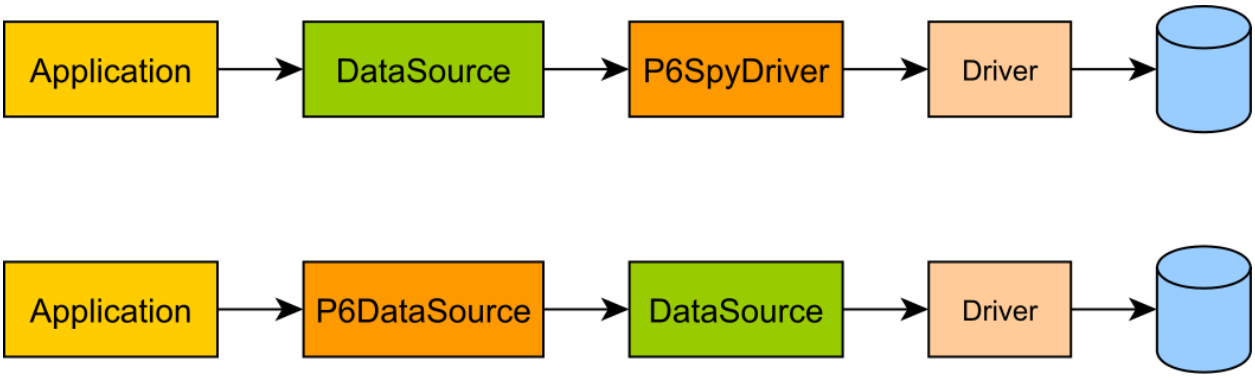

Figure 8.5: P6Spy Architecture

[10] https://github.com/p6spy/p6spy

Running the previous example gives the following output (formatting was also applied):

```
p6spy - 1448122491807|0|batch|connection 7|
        insert into post (title, version, id) values (?, ?, ?)|
        insert into post (title, version, id) values ('Post no. 0', 0, 0)
p6spy - 1448122491807|0|batch|connection 7|
        insert into post (title, version, id) values (?, ?, ?)|
        insert into post (title, version, id) values ('Post no. 1', 0, 1)
p6spy - 1448122491807|0|batch|connection 7|
        insert into post (title, version, id) values (?, ?, ?)|
        insert into post (title, version, id) values ('Post no. 2', 0, 2)
p6spy - 1448122491812|5|statement|connection 7|
        insert into post (title, version, id) values (?, ?, ?)|
        insert into post (title, version, id) values ('Post no. 2', 0, 2)
```

In the order of their occurrence, the output is built out of the following columns:

Table 8.5: P6Spy output

Field	Description
Timestamp	The statement execution timestamp
Execution time	The statement execution duration (in milliseconds)
Category	The current statement category (e.g. statement, batch)
Connection	The database connection identifier (as assigned by P6Spy)
Original statement	The original statement that was intercepted by P6Spy
Formatted statement	The statement with all parameter placeholders replaced with the actual bind values

The first three lines are associated with adding statements to the batch, while the fourth line is logging the actual batch execution (which also explains the *execution time* column value).

One very useful configuration is the `outagedetection` property, which can detect long-running statements.

9. Mapping Types and Identifiers

JPA addresses the Object/Relational mismatch by associating Java object types to database structures Assuming there is a `task` database table having four columns (e.g. `id`,`created_by`, `created_on`, and `status`), the JPA provider must map it to Domain Model consisting of two Java class (e.g. `Task` and `Change`).

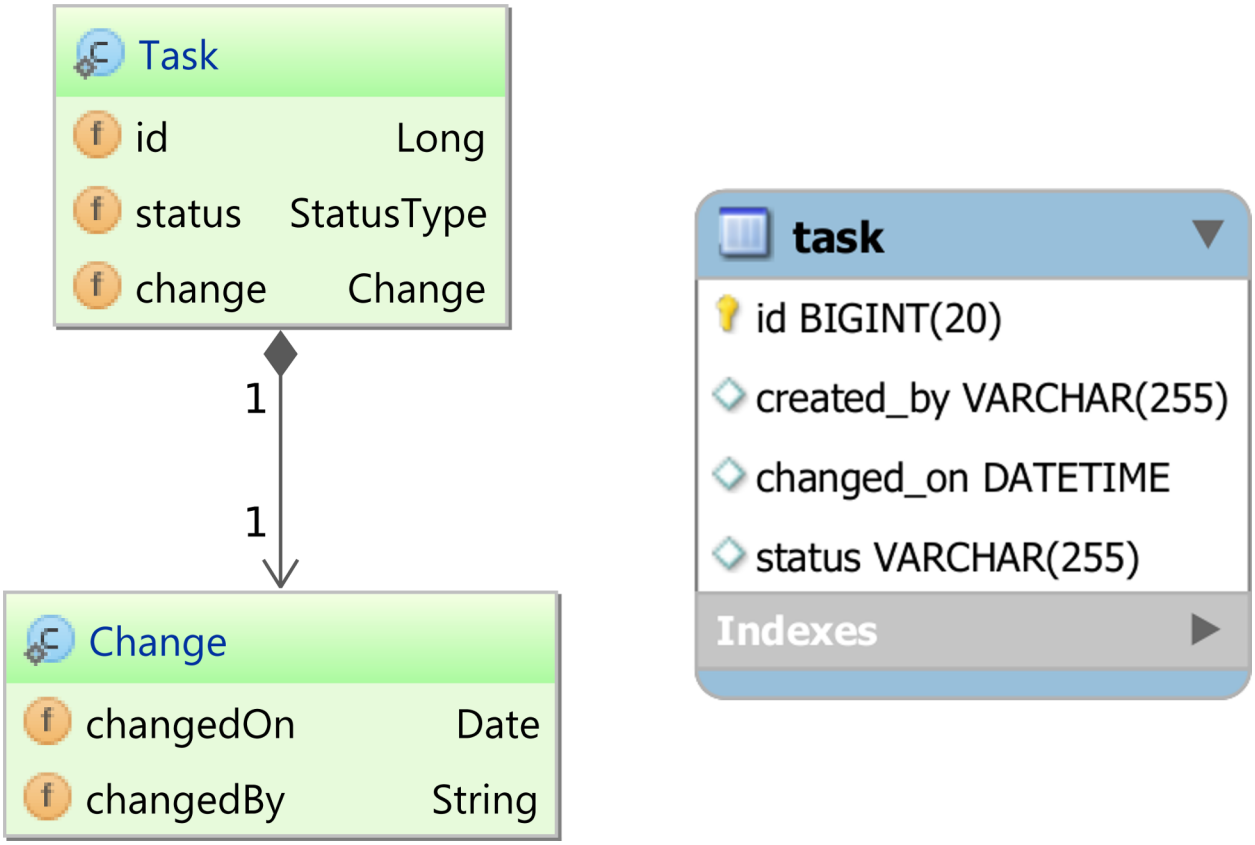

Figure 9.1: Type Mapping

JPA uses three main Object-Relational mapping elements: *type*, *embeddable*, and *entity*. In the previous diagram, the `Task` object is an entity, while the `Change` object is an embeddable type.

Both the entity and the embeddable group multiple Domain Model attributes by relying on Hibernate `Type(s)` to associate database column types with Java value objects (e.g. `String`, `Integer`, `Date`). The major difference between an entity and an embeddable is the presence of an identifier, which is used to associate a database table unique key (usually the primary key) with a Domain Model object attribute.

Although it is common practice to map all database columns, this is not a strict requirement. Sometimes it is more practical to use a root entity and several sub-entities, so each business case fetches just as much info as needed (while still benefiting from entity state management).

Identifiers are mandatory for entity elements, and an embeddable type is forbidden to have an identity of its own. Knowing the database table and the column that uniquely identifies any given row, Hibernate can correlate database rows with Domain Model entities.

An embeddable type groups multiple attributes in a single reusable component.

```
@Embeddable
public class Change {
    @Column(name = "changed_on")
    private Date changedOn;

    @Column(name = "created_by")
    private String changedBy;
}
```

The Domain Model can share state between multiple entities either by using inheritance or composition. Embeddable types can reuse state through composition.

The composition association[1], defined by UML, is the perfect analogy for the relationship between an entity and an embeddable. When an entity includes an embeddable type, all its attributes become part of the owner entity.

The embeddable object cannot define its own identifier because otherwise, the entity has more than one identities. Lacking an identifier, the embeddable object cannot be managed by a Persistence Context, and its state is controlled by its parent entity.

Because they can have a significant impact on the overall application performance, the rest of the chapter discusses types and identifiers in greater detail.

[1] https://en.wikipedia.org/wiki/Object_composition